

Object-Oriented Programming (OOPs) in C# | OOPs Concept in C#

In this article, I will give an overview of Object-Oriented Programming (OOPs) in C#, i.e., discuss the OOPs Principles in C#. Object-Oriented Programming, commonly known as OOPs, is a technique, not a technology. It means it doesn't provide any syntaxes or APIs; instead, it provides suggestions to design and develop objects in programming languages. As part of this article, we will cover the following OOP concepts in C#.

1. **What are the Problems of Functional Programming?**
2. **How can we Overcome the Functional Programming Problem?**
3. **What Is Object-Oriented Programming in C#?**
4. **What are the OOP principles?**
5. **Why do we need Object Oriented Programming in C#?**
6. **Why do we need real-world objects in a Project?**
7. **What types of programming languages come under the OOP system?**
8. **Advantages and Disadvantages of Object-Oriented Programming in C#**

How do we Develop Applications?

Object-Oriented Programming is a strategy that provides some principles for developing applications or software. It is a methodology. Like OOPs, other methodologies exist, such as Structured Programming, Procedural Programming, or Modular Programming. But nowadays, one of the well-known and famous styles is Object Orientation, i.e., Object-Oriented Programming.

Nowadays, almost all the latest programming languages support object orientation. This object orientation is more related to the designing of software, and this deals with the internal design of the software, not the external design. So, it is nowhere related to the users of the software. It is related to the programmers who are working on developing software.

With the help of Object Orientation, application development or programming becomes more and more systematic, and we can follow engineering procedures to develop software. Like in other engineering, how a product is developed, in the same way, a software product is developed by adopting object orientation.

If we talk a little bit about other engineering, like a civil engineer constructing a building, then first of all, he/she will make a plan or design. While making a design or plan, they may have many options, but they will select and finalize one of the designs. Then, they will start constructing once it is finalized as a blueprint on paper. In the same way, an electronic engineer, when manufacturing any device, will come up with some design that is the circuit design of that device on paper. And once that design or blueprint is finalized, he will start manufacturing the device.

So, the **Object Orientation** all depends on how we see the internal system or understand the internal system. So, if you understand the system ideally and if your perspective is very clear, you can develop a better system.

Object-Oriented vs Modular Programming

Now, I will explain to you Object Orientation by comparing it with Modular Programming. The reason is that people who came to learn C# already know the C language. The C programming language supports Modular

or Procedural Programming. Based on that, I can give you an idea of how object orientation differs from modular programming. Let us compare Object-Oriented vs Modular Programming through some examples.

So first, we are taking an example of a bank. If you're developing an application for a bank using modular programming, how do you see the system, how do you see the workings of a bank, and what will be your design? That depends on how you understand it and how you see the system. So, let us see how we look at the bank system using modular programming.

In a bank, you can open an account, you can deposit an amount, you can withdraw an amount, you can check your account balance, or you can also apply for a loan, and so on. So, these are the things that you can do at the bank.

So, **Opening an Account, Depositing Money, Withdrawing Money, Checking Your Balance, and Applying For a Loan** are functions. All these are nothing but functions. And you can do the specific operations by calling that specific function. So, if you're developing software for a bank, it is nothing but a collection of functions. So, the bank application will be based on these functions, and the user of the bank application will be utilizing these functions to perform his required task. So, you will develop software as a set of functions in Modular Programming.

Now, for Object Orientation, we would take some different examples. The government provides many services like electric, water supply, education, and transport, and even the government can have banks. So, these are the different departments of a government. Now, what can you do in the electric department as a user? You can apply for a new connection, you can close your connection if you have extra connections, or you can make a bill payment. What are these? These are functions belonging to the Electric Department.

Now, in the same way, the bank is also there. The same functions like account opening, deposit, withdraw, check balance, apply for a loan, etc., are also there. These are functions belonging to the Bank.

What do we call these? We call them objects. So, the complete system for the government or a complete software for a government is a collection of objects. Now, each object has its relevant functions. So, complete software is a collection of objects containing functions and data related to those functions.

In Modular Programming, the system was a collection of functions. So, if you compare them now, in modular programming, we are looking at the very close level, and in object-oriented programming, we are looking at a little far away level.

Why Object Orientation?

Let us talk about a manufacturing firm which manufactures cars or vehicles. If you look at that manufacturing firm, then it may be working in the form of departments like one is an inventory department that maintains the stock of raw materials and one is manufacturing, which is the production work that they do, and one department will be looking at sales and one department is looking at marketing. One is about payroll, and one is for accounts, and so on. So, there may be many departments.

Suppose you are developing software only for payroll or inventory purposes. In that case, you may look at the system just like a modular approach, and in that, you can find functions like placing an order and checking the item in stock. These types of things can have a set of functions so that you can develop the software only for the inventory system as a collection of functions. Still, when developing software for the entire organization, you must see things in objects.

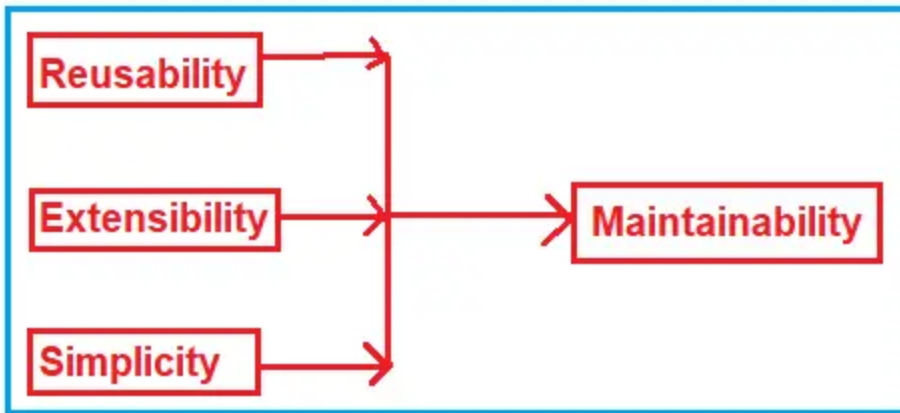
So, the inventory item is an object, an employee is an object, an account is an object, and a product manufacturer is an object. The machines used for production are an object. So, all these things are objects.

Here, you need to see things in the form of objects and define their data and the functions that they're performing. We are looking at the system at a higher level. So, we can adopt object orientation.

What are the Problems of Modular Programming?

Modular programming has the following problems.

1. Reusability
2. Extensibility
3. Simplicity
4. Maintainability



Reusability: In Modular Programming, we must write the same code or logic at multiple places, increasing code duplication. Later, if we want to change the logic, we must change it everywhere.

Extensibility: It is not possible in modular programming to extend the features of a function. Suppose you have a function and you want to extend it with some additional features; then it is not possible. You have to create an entirely new function and then change the function as per your requirement.

Simplicity: As extensibility and reusability are impossible in Modular Programming, we usually end up with many functions and scattered code.

Maintainability: As we don't have Reusability, Extensibility, and Simplicity in modular Programming, it is very difficult to manage and maintain the application code.

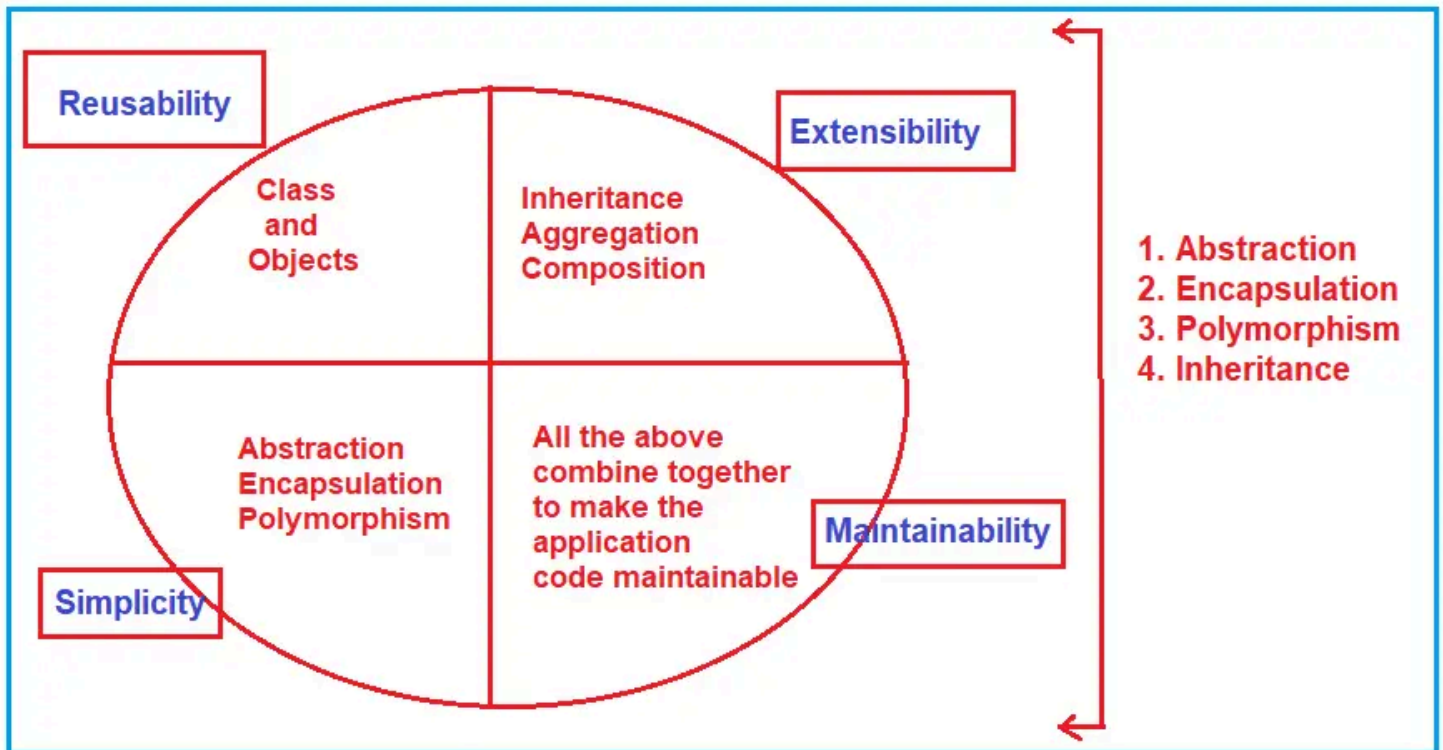
How Can We Overcome Modular Programming Problems?

We can overcome the modular programming problems (Reusability, Extensibility, Simplicity, and Maintainability) using Object-Oriented Programming. OOPs provide some principles, and using those principles, we can overcome Modular Programming Problems.

What Is Object-Oriented Programming?

Let us understand Object-Oriented Programming, i.e., OOP concepts using C#. Object-oriented programming (OOPs) in C# is a design approach where we think in terms of real-world objects rather than functions or methods. Unlike procedural programming language, in OOPs, programs are organized around

objects and data rather than action and logic. Please have a look at the following diagram to understand this better.



Reusability:

To address reusability, object-oriented programming provides something called Classes and Objects. So, rather than copy-pasting the same code repeatedly in different places, you can create a class and make an instance of the class, which is called an object, and reuse it whenever you want.

Extensibility:

Suppose you have a function and want to extend it with some new features that were impossible with functional programming. You have to create an entirely new function and then change the whole function to whatever you want. OOPs, this problem is addressed using concepts called Inheritance, Aggregation, and Composition. In our upcoming article, we will discuss all these concepts in detail.

Simplicity:

Because we don't have extensibility and reusability in modular programming, we end up with lots of functions and scattered code, and from anywhere we can access the functions, security is less. In OOPs, this problem is addressed using Abstraction, Encapsulation, and Polymorphism concepts.

Maintainability:

As OOPs address Reusability, Extensibility, and Simplicity, we have good, maintainable, and clean code, increasing the application's maintainability.

What are the OOPs Principles or OOPs Concepts in C#?

OOPs provide 4 principles. They are

1. **Encapsulation**
2. **Inheritance**
3. **Polymorphism**
4. **Abstraction**

Note: Don't consider Class and Objects as OOPs principle. We use classes and objects to implement OOP Principles.

Let's understand the definitions of the OOPs Principle in this session. From the next article onwards, we will discuss all these principles in detail using some real-time examples.

What are Abstraction and Encapsulation?

The process of representing the essential features without including the background details is called **Abstraction**. In simple words, we can say that it is a process of defining a class by providing necessary details to the external world, which are required by hiding or removing unnecessary things.

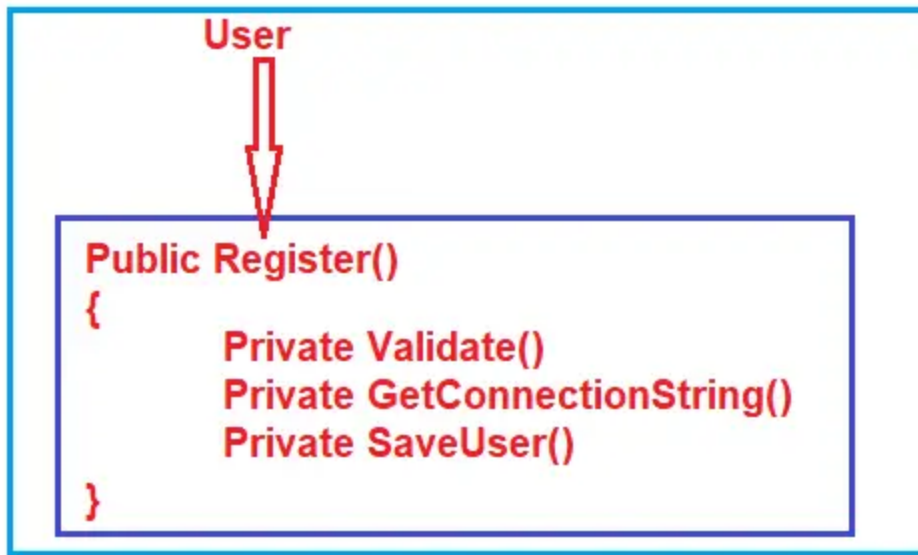
The process of binding the data and functions together into a single unit (i.e., class) is called **Encapsulation**. In simple words, we can say that it is a process of defining a class by hiding its internal data members from outside the class and accessing those internal data members only through publicly exposed methods or properties. Data encapsulation is also called data hiding because, using this principle, we can hide the internal data from outside the class.

Abstraction and Encapsulation are related to each other. We can say that Abstraction is logical thinking, whereas Encapsulation is its physical implementation.

Understanding Abstraction and Encapsulation with an Example:

Let us understand Abstraction and Encapsulation Principles with an example. Suppose you want to design one class for providing the register functionality of a user. For that, what you need to do is first you need to get the data and validate the data, then you need to get the connection string for the database, and finally, you need to save the data in the database. And for this, you have three methods, i.e., Validate, GetConnectionString, and SaveUser. If you provide access to these three methods to the users of this class, then he may end up calling these methods in the wrong order, or it may be possible that he may forget to call any of these methods.

So, here, you need to create one method called Register, and as part of that method, you need to call all these methods (Validate, GetConnectionString, and SaveUser) in a proper sequence. Finally, you need to give access to the Register method instead of the Validate, GetConnectionString, and SaveUser methods. This is what we discussed is nothing but abstraction. How you implement this is nothing but encapsulation. So, here, you need to create the Validate, GetConnectionString, and SaveUser methods with a private access specifier so the user cannot access these methods. Make the Register method Public so the user can access this method, as shown below.



We can achieve code **Simplicity** through Encapsulation and Abstraction.

What is Inheritance?

The process by which the members of one class are transferred to another class is called **inheritance**. The class from which the members are transferred is called the Parent/Base/Superclass, and the class that inherits the Parent/Base/Superclass members is called the Derived/Child/Subclass. We can achieve code **extensibility** through inheritance.

What is Polymorphism?

Polymorphism is derived from the Greek word, where Poly means many and morph means faces/behaviors. So, the word polymorphism means the ability to take more than one form. Technically, we can say that the same function/operator will show different behaviors by taking different types of values or with a different number of values, called **Polymorphism**. There are two types of polymorphism.

1. Static polymorphism/compile-time polymorphism/Early binding
2. Dynamic polymorphism/Run time polymorphism/Late binding

Static polymorphism is achieved by **function overloading and operator overloading**, whereas dynamic polymorphism is achieved by **function overriding**.

Why do we need Object-Oriented Programming (OOPs) in C#?

If you want to represent real-world objects in a programming language for automating the business by achieving **Reusability, Extensibility, Simplicity, and Maintainability**, then you need the concept of OOPs. OOPs provide some principles, and by using those principles, we can develop real-world objects in a programming language by achieving Reusability, Extensibility, Simplicity, and Maintainability.

All living and non-living things are considered objects. So real-world objects such as Persons, Animals, Bikes, Computers, etc., can be developed in object-oriented programming languages by using the OOPs concept.

Why do we need real-world objects in a Project?

We need real-world objects in a project because real-world objects are part of our business. As we are developing applications (software) for automating the business, we must create business-related real-world objects in the project.

For example, to automate the Bank business, we must create real-world objects like Customers, Managers, Clerks, Office Assistants, Marketing Executives, Computers, Printers, Chairs, tables, etc. So, along with the Bank object, we must also create all the above objects because, without all the above objects, we cannot run a Bank business. Technically, we call the above objects business objects.

What types of programming languages come under the OOP system?

The programming languages that implement all the four principles provided by OOPs are called object-oriented programming languages, for example, Java, C#, C++, etc.

Advantages of OOP in C#:

Here's a summary of the key advantages and disadvantages of using OOP concepts in C#:

- **Modularity:** OOP promotes modular code by encapsulating data and behavior within classes. This makes managing and maintaining large codebases easier since you can work on individual classes independently.
- **Code Reusability:** Inheritance allows you to create new classes by reusing existing ones (base classes). This reduces code duplication and promotes a “write once, use many times” approach.
- **Abstraction:** OOP allows you to create abstract classes and interfaces that define a contract without providing implementation details. Abstraction simplifies complex systems by hiding unnecessary details.
- **Polymorphism:** Polymorphism enables you to write code that can work with objects of different classes through a common interface or base class. This flexibility simplifies code and allows for extensibility.
- **Encapsulation:** Encapsulation restricts access to the internal state of objects, promoting data integrity and security. You can control access to fields and methods using access modifiers like public, private, protected, and internal.
- **Maintenance and Debugging:** OOP's modular and organized structure makes debugging and maintaining code easier. Changes in one class typically have limited impact on other parts of the code.
- **Scalability:** OOP principles can help create scalable and extensible software systems. New features can often be added by creating new classes and extending existing ones without modifying the existing code.
- **Readability:** OOP promotes code that is more human-readable and self-explanatory. Classes and objects map closely to real-world entities and interactions, making it easier for developers to understand the code's purpose.

Points to Remember:

1. Object-oriented programming Principles or OOP Concepts in C# are design principles that suggest how we should develop a program so that we can reuse it from other layers of the project effectively and with high scalability.
2. Scalability means we have to develop the project in such a way that it should accept future changes without making major changes to the project. Small changes also should be accepted from external

files like properties files, XML files, etc. Scalability is achieved by developing classes by integrating them in a loosely coupled way.

3. We should develop the project with scalability as there will be business growth. Due to the business growth, we must add required changes to the project with minimal modifications.
4. As a developer, we must remember that in the initial stage of business, the customer never makes a significant investment. As the business grows, customers increase investment according to the growing requirements added to the projects. To add those new requirements, we should not design the project entirely.
5. So, we must strictly design the project by following OOP principles, even though they are not needed at the initial stage, but for accepting future changes.

In the next article, I will discuss the **Class and Objects in C#** with Examples. In this article, I explain the basics of **Object-Oriented Programming (OOPs) in C#**. I hope you enjoy the OOPs Concept in the C# article.